

Arytmetyka Komputerowa

Metody obliczeniowe w nauce i technice

Adrianna Grobel

11.03.2026

Dane techniczne

- System operacyjny - macOS
- Procesor - Apple M3
- Pamięć RAM - 16 GB
- Język programowania: C++20
- Kompilator: g++
- Biblioteki: <iostream>, <iomanip>, <fstream>
- Wizualizacja: Python 3.13.5 (numpy, matplotlib)

Treść zadania

W dokładnej arytmetyce ciąg

$$x_{k+1} = \frac{111 - (1130 - \frac{3000}{x_{k-1}})}{x_k}, x_0 = \frac{11}{2}, x_1 = \frac{61}{11}$$

jest rosnący i zbieżny do 6.

Obliczyć x_{34} dla zmiennych typu float, double, long double i spróbować wyjaśnić uzyskane wyniki.

Dokładna wartość, zaokrąglona do 4 cyfr znaczących, jest równa $x_{34} = 5.998$.

Sposób rozwiązania

```
float f_x_prev = 11.0f / 2.0f;
float f_x_curr = 61.0f / 11.0f;
float f_x_next;

double d_x_prev = 11.0 / 2.0;
double d_x_curr = 61.0 / 11.0;
double d_x_next;

long double ld_x_prev = 11.0L / 2.0L;
long double ld_x_curr = 61.0L / 11.0L;
long double ld_x_next;
```

```
for (int k = 2; k <= 34; ++k) {
    // obliczenia dla float
    f_x_next = 111.0f - (1130.0f - 3000.0f / f_x_prev) / f_x_curr;
    f_x_prev = f_x_curr;
    f_x_curr = f_x_next;

    // obliczenia dla double
    d_x_next = 111.0 - (1130.0 - 3000.0 / d_x_prev) / d_x_curr;
    d_x_prev = d_x_curr;
    d_x_curr = d_x_next;

    // obliczenia dla long double
    ld_x_next = 111.0L - (1130.0L - 3000.0L / ld_x_prev) / ld_x_curr;
    ld_x_prev = ld_x_curr;
    ld_x_curr = ld_x_next;
```

Tabela wyników

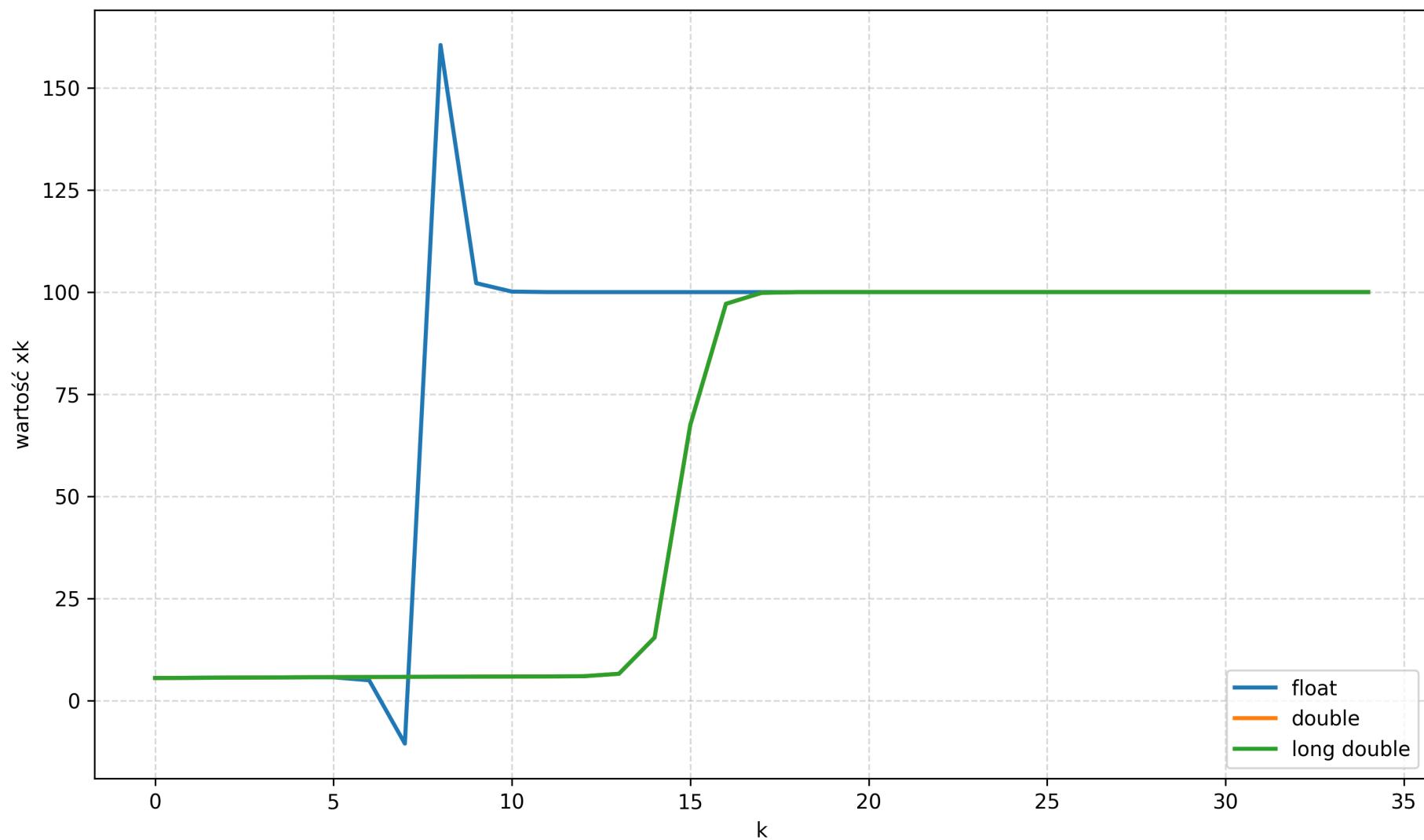
k	float	double	long_double
0	5.5	5.5	5.5
1	5.54545	5.54545	5.54545
2	5.59016	5.59016	5.59016
3	5.63329	5.63343	5.63343
4	5.67205	5.67465	5.67465
5	5.66753	5.71333	5.71333
6	4.94125	5.74912	5.74912
7	-10.5622	5.78181	5.78181
8	160.504	5.81131	5.81131
9	102.19	5.83766	5.83766
10	100.125	5.86108	5.86108
11	100.007	5.88354	5.88354
12	100	5.93596	5.93596
13	100	6.53442	6.53442
14	100	15.413	15.413
15	100	67.4724	67.4724
16	100	97.1372	97.1372
17	100	99.8247	99.8247
18	100	99.9895	99.9895
19	100	99.9994	99.9994

k	float	double	long_double
20	100	100	100
21	100	100	100
22	100	100	100
23	100	100	100
24	100	100	100
25	100	100	100
26	100	100	100
27	100	100	100
28	100	100	100
29	100	100	100
30	100	100	100
31	100	100	100
32	100	100	100
33	100	100	100
34	100	100	100

Tabela 1: Wartości k-tego wyrazu ciągu obliczone z użyciem różnych precyzji reprezentacji liczb zmiennoprzecinkowych.

Wizualizacja wyników

Rysunek 1: Wizualizacja wartości k-tego wyrazu ciągu obliczonego z użyciem różnych precyzji reprezentacji liczb



Specyfikacja liczb zmiennoprzecinkowych w języku C++

- float:

Rozmiar: 4 bajty

Precyzja: 6 cyfr znaczących

Wartość minimalna: $1.17549e-38$

Wartość maksymalna: $3.40282e+38$

- long double:

Rozmiar: 16 bajty

Precyzja: 18 cyfr znaczących

Wartość minimalna: $3.3621e-4932$ Wartość
maksymalna: $1.18973e+4932$

- double:

Rozmiar: 8 bajty

Precyzja: 15 cyfr znaczących

Wartość minimalna: $2.22507e-308$

Wartość maksymalna: $1.79769e+308$

Komentarz do wyników

Z treści zadania wiemy, że w dokładnej arytmetyce ciąg powinien dążyć do granicy równej 6. Jednakże, jak widać w tabeli wyników oraz na wykresie, żaden z zastosowanych typów danych nie pozwolił na osiągnięcie takiego rezultatu – wszystkie warianty zbiegają ostatecznie do wartości 100.

Wyjaśnienie matematyczne:

Zakładając, że ciąg jest zbieżny do pewnej granicy x , dla bardzo dużych wartości indeksów zachodzi równość:

$$x_{k+1} = x_k = x_{k-1} = x$$

Możemy więc zapisać równanie:

$$x = \frac{111 - (1130 - \frac{3000}{x})}{x}$$

Po pomnożeniu obustronnie przez x^2 i uporządkowaniu, otrzymujemy równanie wielomianowe trzeciego stopnia:

$$x^3 - 111x^2 + 1130x - 3000 = 0$$

Pierwiastkami tego równania są liczby: **5**, **6** oraz **100**.

$$x_1 = \frac{61}{11} = 5.5454 \dots$$

Komputer musi tę liczbę zaokrąglić, co już na starcie wprowadza pewien błąd reprezentacji.

Zastosowany wzór rekurencyjny jest silnie niestabilny numerycznie i z każdą kolejną iteracją błąd jest potęgowany. Ostatecznie całkowicie zagłusza właściwe obliczenia i ściąga ciąg do najsilniejszego atraktora w tym równaniu, czyli liczby 100.

Wnioski

Zadanie to jest dowodem na to, że zwiększanie precyzji typu danych nie rozwiązuje problemu źle uwarunkowanych zadań numerycznych. Użycie typu double zamiast float potrafi jedynie opóźnić ujawnienie się niestabilności o kilka kroków, ale nie jest w stanie zmienić ostatecznego, błędnego wyniku działania algorytmu.

Kluczowe jest więc projektowanie i wybieranie algorytmów stabilnych numerycznie.